P3881R0
Forward-progress for all infinite loops

Published Proposal, 2025-11-05

This version:
https://wg21.link/p3881r0

Authors:
Simon Cooksey (NVIDIA)
Gonzalo Brito Gadeschi (NVIDIA)

Audience:
SG1

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This proposal will provide forward progress to programs reaching infinite loops that do not contain side-effects.

Table of Contents

In C++, infinite loops without side effects are undefined behavior. C++ relies on this to define forward progress for programs without such loops. This also allows compilers to perform optimizations on loops.

These optimizations can change program behavior in surprising ways, and the capability to perform them complicates toolchains, interoperability with Rust, and artificially requires programmers to clutter their code to avoid this UB.

This proposal improves C++ to provide forward progress to programs reaching infinite loops that do not contain side-effects, while preserving the current proress guarantess where infinite loops do contain side-effects.

The main benefits of this proposal are:

- all programs with infinite loops become well-defined, i.e., it removes an entire source of undefined behavior from the C++ specification.
- simplifies the language, its specification, and its implementations.

These benefits are obtained by trading off some implementation flexibility. However, the theoretical optimizations we loose are not being realized in practice.

§ 1. Motivation

In C++, infinite loops without side effects are undefined behavior under the forward-progress rules. This leads to surprising and hazardous consequences: compilers may remove or transform loops in ways that change observable behavior, even enabling latent bugs such as eliminating loops guarding null pointer dereferences. To avoid this, programmers must pepper dummy yield points or volatile operations merely to suppress UB.

Maintaining this UB assumption complicates compiler infrastructure. For example, LLVM `mustprogress` exists solely to encode C++'s unique loop semantics. This causes LLVM to maintain separate progress models for C, C++, and Rust, increasing complexity and risking inconsistent optimizations for little measurable performance benefit. A comprehensive empirical study [Popescu and Lopes, 2025] shows that removing this UB causes negligible regressions in real workloads and even improves performance in some workloads.

Making infinite loops well-defined aligns C++ with Rust (any other language?) enabling consistent compiler optimizations, simplifying interoperability, and reducing hidden pitfalls for developers. It also makes the language easier to reason about, while providing a more predictable foundation for formal specifications.

The trade-off is that implementations lose some flexibility to perform optimizations that speculate loops terminate. However, the study referenced above shows that these opportunities are rarely realized in practice. The gains in safety, clarity, and cross-language and consistency far outweigh the minor theoretical loss of optimization potential.

2. Impact

2.1. Language impact

This is a backward compatible change that simplifies the language specification.

No pre-existing programs are impacted, e.g., the forward progress provided to any pre-existing program remains the same. This proposal just allows new programs, and defines the forward progress guarantees for those programs.

Before this change, this program exhibited undefined behavior:

```
int main(void) {
  bool True = true;
  while(True);
  return 0;
}
```

and after this change it becomes well defined.

No execution of the following example terminates or exhibits UB:

```
int main() {
  while(true);
  *nullptr;
  return 0;
}
```

The consequences for multi-threaded programs is that this program is not guaranteed to abort:

```
#include <atomic>
#include <thread>

std::atomic<bool> flag = false;
```

```
void thread0() {
  flag.store(true);
  abort();
}

void thread1() {
  flag.wait(false);
  bool True = true;
  while(True);
}

int main() {
  std::jthread t0(thread0);
  std::jthread t1(thread1);
  return 0;
}
```

After observing the flag change, thread 1 will never reach an execution step, degrade the forward progress afforded to all threads (including thread0) to *weakly parallel forward progress*, the implementation is not reqruired to ensure that execution reaches the abort.

§ 2.2. Implementation impact

The impact on clang is restricted to stop emitting the `mustprogress` attribute when compiling C++, in the same way it does not emit it when compiling C (godbolt example). Removing `mustprogress` from LLVM has the potential to significantly simplify LLVM internals because this matters for tricky optimizations, and validating them twice is harder than once, and right now we are not confident either of the paths are always correct.

GCC already appears to implement this correctly for simple cases (see here).

§ 2.3. Performance impact

LLVM IR mustprogress attribute and metadata (llvm.loop.mustprogress) exists exclusively to exploit the UB allowed by C++'s intro.progress.1:

> LLVM IR Specification: The mustprogress attribute is intended to model the requirements of the first section of intro.progress of the C++ Standard.

It is incorrect for frontends to use it for other programming languages (C, Rust). It was added in 2020 (0, 1) and made C and Rust progress semantics the default in LLVM. C++ semantics became opt-in, which allowed LLVM to properly implement C and Rust without blocking that effort on evaluating any potential impact on C++ (such impact was not well understood at the time).

In PLDI 2025, Lucian Popescu and Nuno P. Lopes presented Exploiting Undefined Behavior in C/C++ Programs for Optimization: A Study on the Performance Impact (PLDI'25), which--among many others--includes an experiment (FD3) that evaluates the impact of this change on 24 of the Phoronix benchmarks:

> Flag FD3 prevents the compiler from assuming that all loops without side-effects terminate. This amounts to removing LLVM's mustprogress attribute from functions.

Figure 2 of that paper shows that with LTO enabled, there are no severe performance regressions from this change on Arm, AMD or Intel CPUs. The % of moderate regressions on Arm is 0%, on Intel is 4% and on AMD is 8%. The % of performance improvements on Arm is 88%, on Intel is 13%, and on AMD is 4%.

We did not expect that disabling this optimization would improve performance and feel that further analysis is required to better understand the performance impact of this proposal.

The following example shows how this change impacts compiler optimizations. Before this proposal, this program:

```
int c = 0;
while(1) {
  if (c == 42)
    break;
  c = foo(); // read-mem-only (non volatile, non atomic)
}
return c;
```

can be optimized to:

```
return 42;
```

because UB allows the optimizer to assume the loop always terminates.

After this proposal, the above program can still be optimized to:

```
if (foo() == 42) return 42; // loop removed in finite path
while (1);                  // infinite path must be preserved
unreachable();
```

because there are two execution paths this loop can take:

- a finite one that performs a single loop iteration, matches 42, exits and returns 42, or
- an infinite path that never exits the loop.

This proposal does not prevent optimizing the happy finite path, it just requires preserving the infinite path (which before the compiler was not required to preserve).

§ 3. Wording

§ 3.1. Modify [intro.progress] as follows:

1 ~~The implementation may assume that any thread will eventually do one of the following:~~ The following operations are *execution steps*:

- (1.1) terminate,
- (1.2) invoke the function `std::this_thread::yield` ([thread.thread.this]),
- (1.3) make a call to a library I/O function,
- (1.4) perform an access through a volatile glvalue, or
- (1.5) perform an atomic or synchronization operation other than an atomic modify-write operation ([atomics[.order]) . ~~, or~~
- ~~(1.6) continue execution of a trivial infinite loop ([stmt.iter.general]).~~

If any thread does not eventually perform an *execution step* the implementation is allowed to provide *weakly parallel forward progress* for all threads.

[Note 1: compilers are not allowed to remove empty infinite loops. That is, C++ guarantees that a thread that reaches the following function does not exhibit undefined behavior:

```
// Example 1:
void halt() {
  while(true);
```

```
  *nullptr;
}
```

But when a program reaches `halt()` the implementation is permitted to starve all other threads of execution. Implementations can still reorder code before potentially infinite loops as long as the effect of these optimizations is not observable

```
void foo() {
   std::atomic<int> x = 0;
   while(undecidable()) { math ... }
   x.store(42, memory_order_relaxed);
   // can still be reordered before the loop, so long as the as-if rule ho
}
```

```
void main() {
  std::atomic<int> x(0);

  std::jthread([&]() {
    while(undedicable()) { ... }
    x.store(42, memory_order_relaxed); // may not be re-ordered
  });

  std::jthread([&]() {
    printf("%d", x.load(memory_order_relaxed));
  });

  return 0;
}
```

- end note]

~~[Note 1: This is intended to allow compiler transformations such as removal, merging, and reordering of empty loops, even when termination cannot be proven. An affordance is made for trivial infinite loops, which cannot be removed nor reordered. — end note]~~

3 ~~During the execution of a thread of execution, each of the following is termed an execution step:~~

- ~~(3.1) termination of the thread of execution,~~
- ~~(3.2) performing an access through a volatile glvalue,~~
- ~~(3.3) completion of a call to a library I/O function, or~~

- ~~(3.4) completion of an atomic or synchronization operation other than an atomic modify-write operation ([atomics.order]).~~

5 ~~[Note 4: Because of this and the preceding requirement regarding what threads of execution have to perform eventually, it follows that no thread of execution can execute forever without an execution step occurring. — end note]~~